End-to-End Efficiency of a Packed Projected W1X1/N1 CIFAR-10 ViT

Reviewer-response insert: trained-model accuracy, architecture, precision allocation, projection cost, latency, throughput, memory, energy, full-precision ablation, and hardware scaling

Reviewer question. "Provide actual efficiency measurements (latency, throughput, memory, energy, or bit-ops) for a representative ViT, accounting for projection expansion and all full-precision modules. This is the decisive item."

Response in one sentence. For a complete trained 12-block CIFAR-10 ViT on an H100 at batch 1, the packed structured W1X1/N1 implementation retains **85.70% test accuracy**, is **1.472× faster**, delivers **1.472× higher throughput**, consumes **31.5% less gross device energy per image**, uses **13.13× less persistent tensor state**, and is **24.74× smaller on disk** than the recorded FP32 training checkpoint; the measured path includes projection, packing, all attention/MLP blocks, every norm and residual, patch embedding, and the classifier.

Evaluated topology and precision map. The trained model has 65 tokens (64 patches plus one special token), width $d = 192$, 12 Transformer blocks, six 32-wide heads, a $192 \to 768 \to 192$ OddGate MLP, and ten output classes. Every block's six logical body matrices—$W_Q, W_K, W_V, W_O, W_{\text{up}}, W_{\text{down}}$—use projected one-bit weights and projected one-bit activations. The projection factor is $N/d = 1$; therefore the experiment has no code-length expansion.

Component	Stored precision	Runtime treatment
72 logical body matrices	$Q(GW)$, one bit/coordinate	48 physical packed modules because Q/K/V share one concatenated code tensor; XOR–popcount plus FP32 row gains
Projection G	two seed recipes in the artifact	four regenerated int8 diagonals (1,920 bytes total); two-pass block-32 Hadamard; dense G is never created
Projected activations	not persistent	$Q(Gx)$ is ballot-packed and consumed on-chip; no expanded Gx global tensor
Residual stream / attention	transient BF16	BF16 block boundaries; retained Flash/SDPA score–softmax–value computation
Retained model state	BF16 / FP32	patch embed, positions, norms, temperatures, residual coefficients, row gains, final norm/head

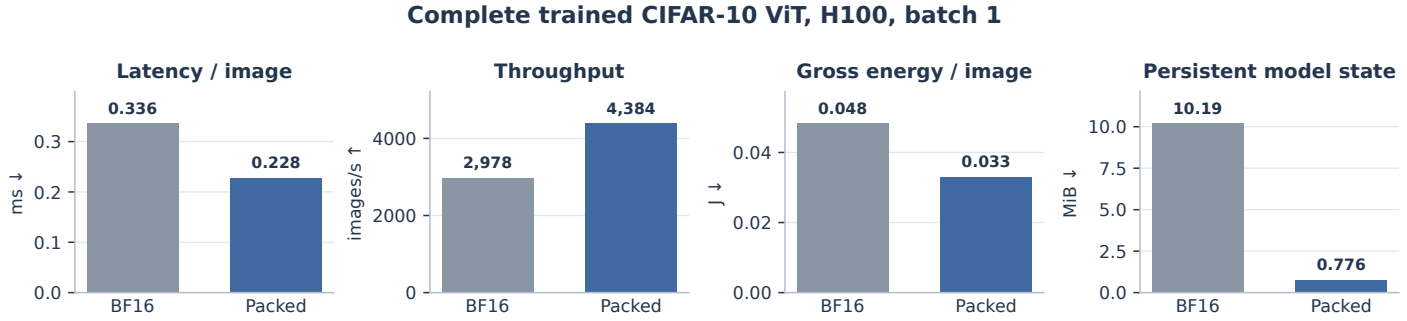

Figure 1. Measured complete-model H100 result. Lower is better for latency, energy, and state; higher is better for throughput.

Headline protocol and result. NVIDIA H100 80 GB HBM3; PyTorch 2.12.0+cu126; CUDA 12.6; preloaded inputs; 25 warm-ups; 800 CUDA-event samples; static-shape CUDA Graphs; separate eight-second NVML loops under a shared measured 121.30 W idle power. The BF16 comparator has the same 12×192 topology, TorchInductor-compiled body, fused QKV/SDPA, and the identical fused pool/LayerNorm/head. Median latency is 0.335776 versus 0.228096 ms/image; throughput is 2,978.2 versus 4,384.1 images/s. Gross energy is 0.048241 versus 0.033058 J/image (1.459×, or 31.5% lower); idle-adjusted energy is 0.007801 versus 0.005697 J/image (1.369×, or 27.0% lower).

Accuracy and comparator boundary. The deployed packed artifact independently obtains 85.70% over all 10,000 CIFAR-10 test images. The preserved training record reports 90.34% for its FP teacher and 85.51% for the hard structured student. A fixed-logit audit gives maximum absolute error 0 and 100% top-1 agreement against the canonical packed reference. The speed comparator uses random BF16 body values because the deployable artifact intentionally contains no latent FP body; values do not change these static-shape kernel timings.

Why the memory reduction is real—and what is still full precision

Reviewer question. "If projection matrices are regenerated, are the original FP weights and activations still stored and communicated? Would this not exceed the FP baseline, especially when activation traffic dominates?"

Training and deployment are different regimes.

- **During QAT**, latent FP weights and optimizer state may be retained. We make no training-memory reduction claim.
- **During deployment**, latent W is absent. Each body matrix is represented only by packed $Q(GW)$ bits and one FP32 gain per output row. No optimizer state, latent body, or dense G is present.
- **G is not regenerated as a dense matrix.** Two versioned SplitMix64 seed recipes deterministically create the two pairs of sign diagonals used by the width-192 and width-768 block-Hadamard operators. Their shared CUDA cache is 1,920 bytes and is explicitly included below.
- **The residual stream remains BF16.** We do not claim a persistently one-bit network state between every operation. Projected signs are ephemeral, formed and consumed inside the low-bit kernels; attention context remains BF16.

Let $P_{\text{body}} = 5{,}308{,}416$ be the number of body weights. A BF16 body uses

$$M_{\text{body}}^{\text{BF16}} = 2P_{\text{body}} = 10{,}616{,}832 \text{ bytes} = 10.125 \text{ MiB},$$

whereas its packed signs require $P_{\text{body}}/8 = 663{,}552$ bytes (0.633 MiB). Adding all retained state gives

$$\boxed{M_{\text{BF16}}^{\text{persistent}} = 10.189 \text{ MiB}, \quad M_{\text{packed}}^{\text{persistent}} = 0.776 \text{ MiB}, \quad \text{compression} = 13.13\times .}$$

The execution layout also keeps a transposed 0.633 MiB code view for coalesced word-major loads. Counting that duplicate and the 1,920-byte projection cache gives 1.411 MiB of total resident packed tensor state, still **7.22**× below BF16.

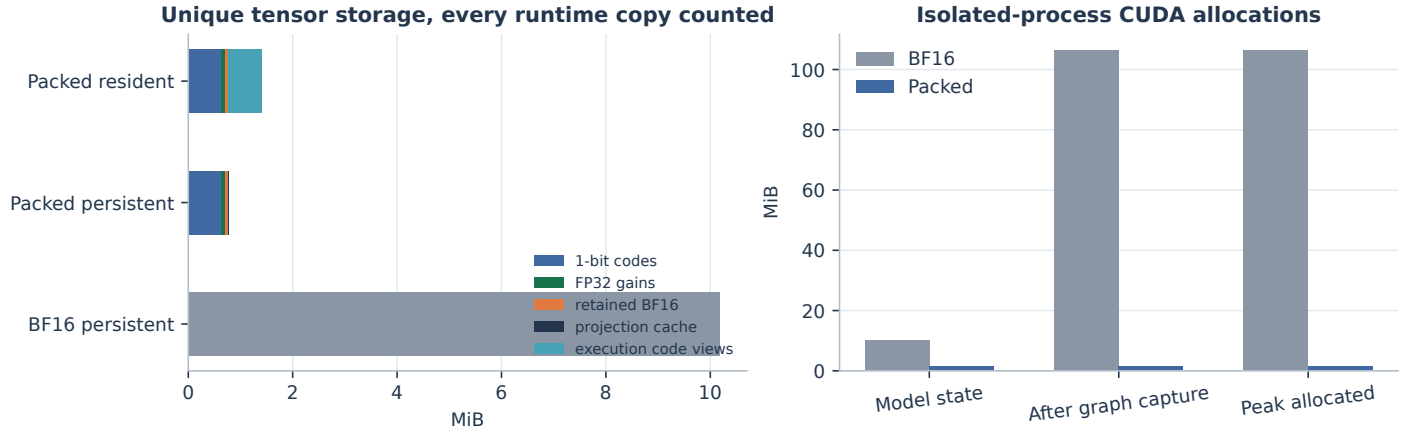

Figure 2. Left: unique-storage accounting explicitly includes the execution-only code view and regenerated projection cache. Right: isolated-process live CUDA tensor allocations at batch 1.

Serialized checkpoint. The deployable `GGDPACK1` file is 872,512 bytes (0.832 MiB): 663,552 bytes of binary codes, 82,944 bytes of FP32 gains, 67,412 bytes of retained state, and manifest/alignment overhead. It is mmap-friendly: a JSON manifest indexes 64-byte-aligned tensors by dtype, shape, offset, category, and SHA-256. The recorded FP32 training `.pth` is 21,583,565 bytes (20.584 MiB), making the actual disk ratio 24.74×. SHA-256 verification covers every payload; the canonical artifact hash is `a2a7cb9fc0bd...a4332031`.

Whole-process versus model-state memory. In isolated processes, graph-captured live CUDA allocations are 106.221 MiB for compiled BF16 and 1.452 MiB for packed; measured peaks are 106.391 and 1.596 MiB. These figures exclude the common CUDA context but include compiler/graph allocations. They should not be confused with the more portable model-state comparison: compiler caches vary by PyTorch/GPU, while the 10.189 versus 1.411 MiB tensor-state accounting is exact unique storage.

What is communicated between layers. One BF16 residual stream is $65\times192\times2 = 24{,}960$ bytes per image and is reused sequentially, not stored once per layer. Q/K/V and attention context are transient BF16. The optimized cross-sublayer kernel nevertheless removes a major BF16 traffic boundary: the attention output is normalized, rounded exactly as before, and retained on-chip through the next complete OddGate MLP. Its 768-wide hidden activation remains in shared memory; $Q(Gx)$ remains in registers/shared memory as packed words.

Actual execution cost, full-precision contribution, and batch ablation

Reviewer question. "Which modules are quantized, how much do retained FP components contribute, and does the gain survive projection expansion and realistic execution overhead?"

Measured CUDA-time attribution. A 50-forward CUDA trace totals 0.224993 ms/image, consistent with the 0.228096 ms headline. Projected low-bit kernels account for **61.17%**: 41.18% for the cross-sublayer attention-output/MLP kernel and 19.99% for packed QKV boundaries. Retained Flash attention accounts for 28.10%; final norm/head 4.57%; token/layout operations 3.63%; and the BF16 patch convolution 2.54%. The Hadamard projections, centering, sign packing, popcounts, gains, OddGate, norms, and residual epilogues are already charged to the two blue categories.

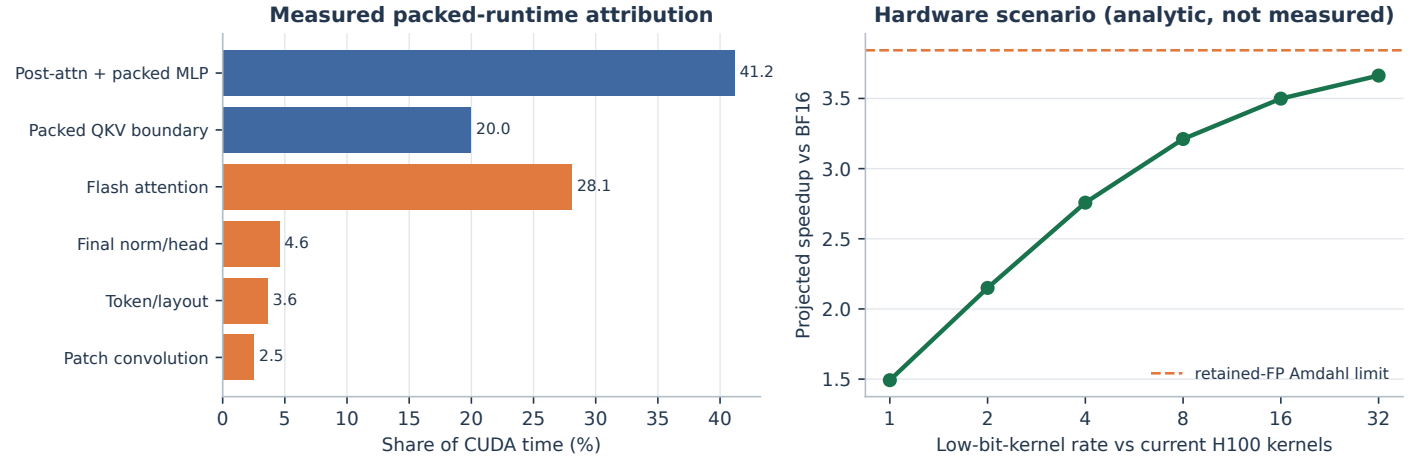

Figure 3. Left: measured complete packed-runtime CUDA attribution. Right: latency-derived Amdahl scenario for a native low-bit engine; the curve is an analytic projection, not a hardware measurement.

Per-image operation class	Count	Precision / implementation
Body sign pair products	345.047 million	one-bit signs; 10.783 million 32-bit XOR–popcount word terms
Structured projection work	10.483 million	add/sub operations; two block-32 Hadamard passes, fully measured
Attention $QK^\top$ and AV	19.469 million MACs	retained BF16 Flash/SDPA
Patch embedding	0.590 million MACs	retained BF16 convolution
Classifier	0.00192 million MACs	retained BF16 fused final boundary

What ImageNet optimization transferred to CIFAR. The old CIFAR implementation wrote the rounded 192-vector after attention to global memory and launched a separate MLP-residual kernel. The new topology-preserving kernel keeps that BF16-rounded vector on-chip and immediately performs MLP pre-norm, width-192 Hadamard/sign packing, packed FC1, OddGate, centered width-768 Hadamard/sign packing, packed FC2, post-norm, and residual addition. Artifact keys and numerical behavior are unchanged. On the same H100 protocol, packed latency falls from 0.248928 to 0.228096 ms (8.4%), moving complete-model speedup from 1.352× to 1.472×. All 20 unit/integration tests pass; the fixed audit remains bit-exact.

Batch/cache ablation. The packed path is faster at batches 1, 2, 4, 8, and 16, with speedups 1.464×, 1.478×, 1.513×, 1.515×, and 1.188× in the saved sweep. At batch 32 the ratio is 0.940×: dense weights are heavily reused and BF16 Tensor Cores are better utilized. Gross energy follows the same operating-region trend and crosses earlier between batches 8 and 16. This is why the claim is explicitly a small-batch latency/energy result, not universal dominance.

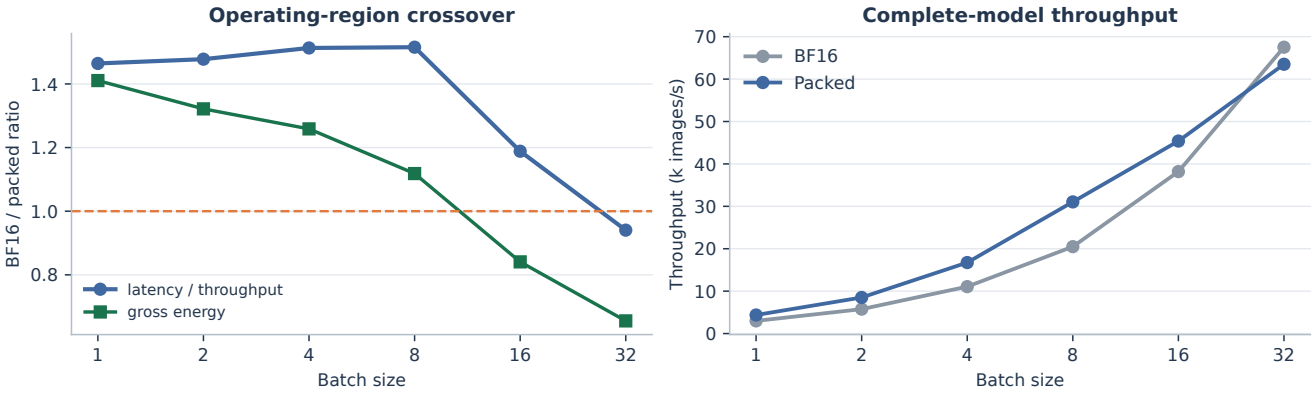

Figure 4. Latest-kernel H100 sweep. Latency values are per batch; throughput and energy are normalized per image.

Why H100 is conservative, expected scaling, and reproducibility

The H100 disadvantage is specific, not rhetorical. H100 makes the comparator unusually strong: mature cuBLAS/cuDNN/Inductor kernels, fourth-generation Tensor Cores, and a 50 MB L2 cache. The entire 10.189 MiB BF16 model state fits in L2 after warm-up. H100 offers fixed binary MMA tiles, but the native $m8n8k128$ interface was slower for this 65-token, width-192 workload once Hadamard projection, packing, gains, norms, and residual epilogues were included. The retained implementation therefore uses shape-specialized integer XOR–popcount rather than an equally mature binary Tensor Core datapath. The measurement still charges all software projection and packing overhead.[1]

Why larger models should move the crossover right. This ∼5.3 M-body-weight model is deliberately stringent: both packed and BF16 bodies are cache-resident. In a model whose BF16 layer state exceeds cache, small-batch inference must stream more weight bytes from HBM, whereas a one-bit body moves roughly one-sixteenth as many code bytes (before gains and FP boundaries). Weight reuse then becomes less favorable to BF16, so the memory-bound low-bit operating region should extend to larger batches. This is a roofline expectation, not a substituted result; larger-model evidence must repeat the same batch sweep.

Expected outcome on low-bit-friendly hardware. A suitable engine would provide packed XNOR/popcount matrix units, packed activation SRAM, an implicit Hadamard/sign front end, and fused FP gain/residual epilogues. The current H100 trace partitions 0.224993 ms into 0.137625 ms projected low-bit kernels and 0.087368 ms retained FP/fixed work. If only the low-bit portion improved by 4×, 8×, or 16×, the transparent Amdahl estimates relative to the measured BF16 baseline are

$$S(4) = 2.76\times, \qquad S(8) = 3.21\times, \qquad S(16) = 3.50\times .$$

The retained-FP ceiling is 3.84× unless Flash attention, patch embedding, and the final boundary are also quantized or fused. These are *latency-derived scenarios*, not fabricated ASIC measurements. Energy is not inferred from bit count: the defensible energy claim is the measured H100 reduction of 31.5% gross and 27.0% idle-adjusted per image.

Claim boundary and reproducibility.

- The result is a representative trained CIFAR ViT system result; it is not relabeled as CNN, ImageNet, or NLP evidence.
- The runtime does not claim training-memory compression. It claims deployable checkpoint, resident inference state, measured inference latency/throughput, and measured device energy.
- Every full-precision component is enumerated and included. The profile exposes its measured latency share rather than hiding it in an analytic bit-op count.
- `runtime/demo.py` reproduces accuracy and the batch-1 headline; `batch_sweep.py`, `profile_components.py`, and `memory_audit.py` reproduce the ablations. Raw JSON/CSV reports, plotting source, 20 tests, and the artifact accompany this PDF.

[1] NVIDIA H100 architecture and 50 MB L2: `https://resources.nvidia.com/en-us-tensor-core/nvidia-hopper-architecture-whitepaper`.

Expansion-factor cost and the structured projection recipe

Reviewer question. "Standard layers compute a p-dimensional inner product, whereas Angular Layers also compute Gx and Gw with $G \in \mathbb{R}^{N\times p}$ and $N = \nu p$. How does this affect training and inference cost for $\nu \in \{1, \dots, 4\}$? Were practical slowdowns observed?"

Construction used by the measured hardware path. Let $b = 32$, let $H_b \in \{-1, +1\}^{b\times b}$ be the unnormalised Walsh–Hadamard matrix ($H_b H_b^\top = bI$), and split the last dimension into p/b blocks. Frame f is

$$G_f x = \mathrm{vec}\left(\frac{1}{\sqrt{b}} H_b D_{2,f} H_b D_{1,f} x^{(j)}\right)_{j=1}^{p/b}, \qquad G = \begin{bmatrix} G_1 \\ \vdots \\ G_\nu \end{bmatrix}, \quad N = \nu p.$$

Each $D_{r,f}$ is a diagonal Rademacher matrix generated deterministically from the versioned SplitMix64 seed streams $2f$ and $2f + 1$. Equal input dimensions share the same recipe. Neither training nor inference stores a dense $N \times p$ matrix. The portable checkpoint stores the seed, frame count, and packed $Q(G_f W)$ codes. At inference $Q(G_f W)$ is precomputed once; online execution forms $Q(G_f x)$ with fused Hadamard/sign ballot and performs packed correlation. Q/K/V reuse the same projected activation.

For a two-pass transform, online projection work is $O(2\nu p \log_2 b)$ add/sub operations per token, while packed correlation contains νmp sign-pair products for an $m \times p$ layer. Weight-code storage is exactly νmp bits before row metadata. Thus expansion is linear in operation count and code storage, even though wall time need not scale linearly because retained FP work and launch overhead are unchanged.

Complete training measurement. The table reports forward, cross-entropy backward, and one SGD update for the full 12-block model at batch 64. Patch embedding, norms, residual arithmetic, attention score/softmax/value products, and classifier are included. The analytical backward does not grow with ν, explaining the modest end-to-end sensitivity.

ν	step (ms)	images/s	change vs $\nu = 1$	peak allocated (MiB)
1	13.693	4,674	—	488.4
2	14.593	4,386	+6.6%	489.5
3	14.644	4,370	+7.0%	488.1
4	14.813	4,321	+8.2%	487.4

The compiled BF16 training comparator is 8.063 ms, so the angular path is 1.70× slower at $\nu = 1$ and 1.84× at $\nu = 4$ on H100. This slowdown is reported explicitly; H100's BF16 Tensor Cores are a particularly strong training baseline.

Complete packed-inference expansion measurement. To isolate expansion, all factors use one portable implementation of the complete ViT with precomputed packed weight codes, TorchInductor around compiler-visible CUDA operators, static CUDA Graphs, 25 warm-ups, 400 CUDA-event samples, and separate eight-second NVML loops. The architecture-specific $\nu = 1$ headline remains 0.228096 ms/image; it is not replaced by the less-fused portable $\nu = 1$ row below. Ratios within this table are the controlled expansion result.

ν	latency (ms)	vs $\nu = 1$	images/s	gross J/image	persistent MiB
1	0.670464	1.000×	1,491.5	0.094756	0.777
2	0.717984	1.071×	1,392.8	0.104415	1.410
3	0.789792	1.178×	1,266.2	0.117368	2.043
4	0.837120	1.249×	1,194.6	0.127364	2.675

Every factor passed an exact QAT-hard-forward versus packed-runtime audit (maximum logit difference 0 and 100% top-1 agreement). From $\nu = 1$ to 4, measured latency rises 24.9%, gross energy/image rises 34.4%, and binary-code storage rises exactly 4×; retained FP state makes total persistent state rise only 3.44×.

Design conclusion. Our reported accuracy experiments therefore default to $\nu = 1$: additional frames reduce estimator variance, but they are not the most cost-effective accuracy knob observed in our studies. Increasing quantizer resolution at fixed $N = p$ often yields a larger accuracy gain because it preserves more information without repeating the structured projection. The cost statement must nevertheless be precise. During QAT, higher bit-width keeps the same matrix dimensions and is often close in wall time; it is not mathematically free. In a bit-serial packed runtime, WkXℓ requires up to $k\ell$ binary correlation planes and k stored weight planes. For example, W2X2/N1 and W1X1/N4 both represent four binary pair-product planes, but W2X2/N1 needs only one structured activation projection. Native multi-bit hardware may execute those planes more efficiently. We therefore claim *same asymptotic low-bit regime and often better accuracy per unit cost*, not identical measured W1X1 latency.

Separation of evidence. The highly fused $\nu = 1$ runtime supplies the headline deployment speed/energy result. The portable factors 1–4 runtime supplies a controlled expansion ratio. Accuracy at $\nu > 1$ requires checkpoints trained with those exact independent frames and is not inferred from this systems sweep.

Canonical artifact. `runtime/artifacts/cifar_vit_hadamard32_w1x1n1.ggdpack`, 872,512 bytes, SHA-256 `a2a7cb9fc0bd01e5...a4332031`.
Headline report. `runtime/results/latest.json`.